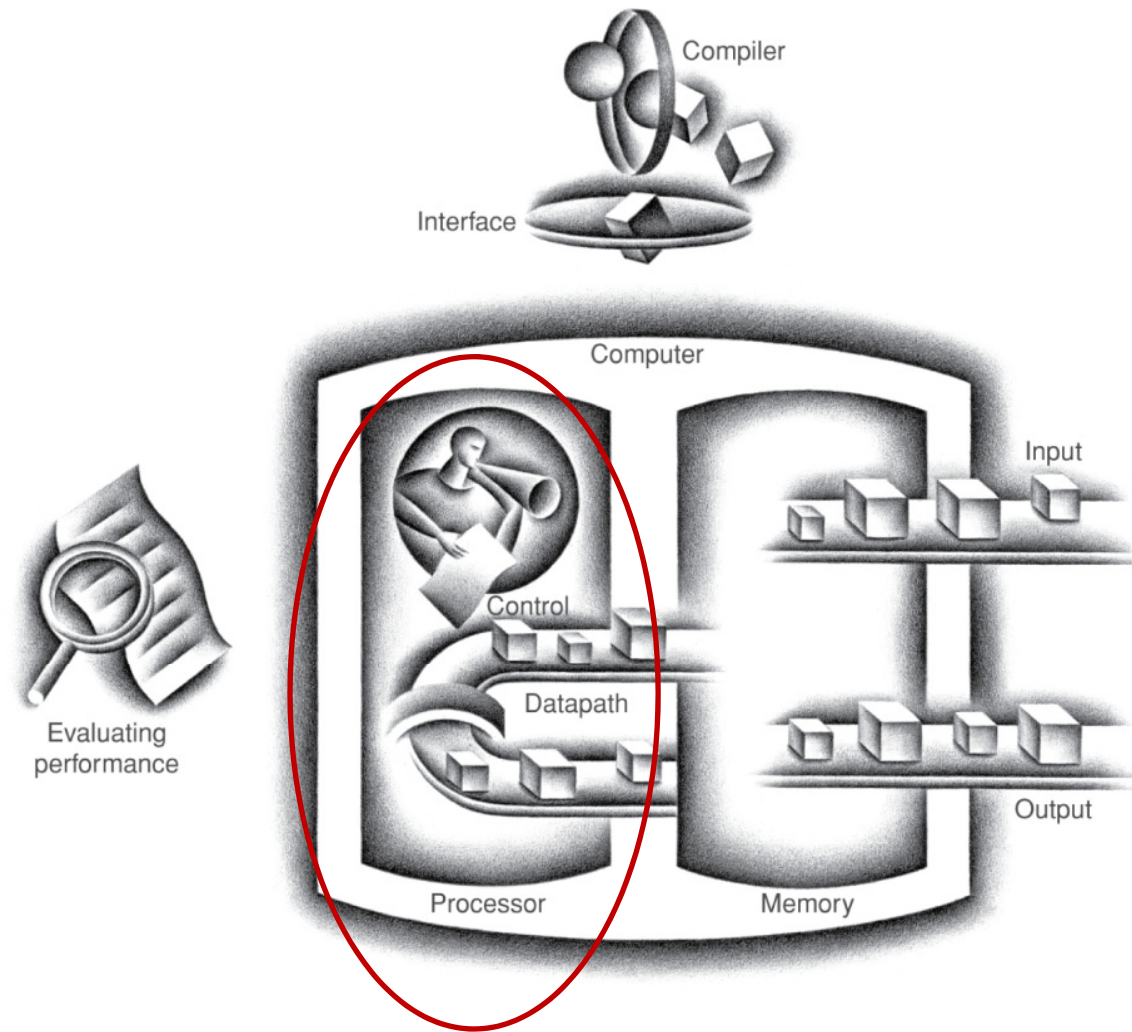


Lecture 4 :Pipelining

Components of a Computer



Processor

- Datapath

- Component of the processor that performs arithmetic operations

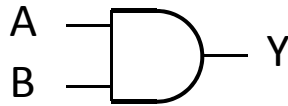
- Control

- Component of the processor that commands the datapath, memory, I/O devices according to the instructions of the memory

A Recap: Combinational Elements

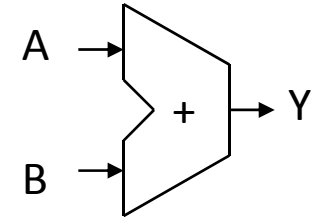
- AND-gate

- $Y = A \& B$



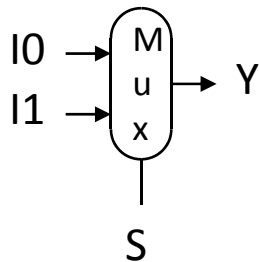
- Adder

- $Y = A + B$



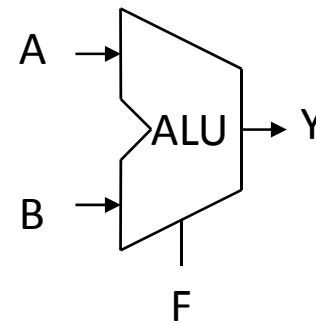
- Multiplexer

- $Y = S ? I1 : I0$



- Arithmetic/Logic Unit

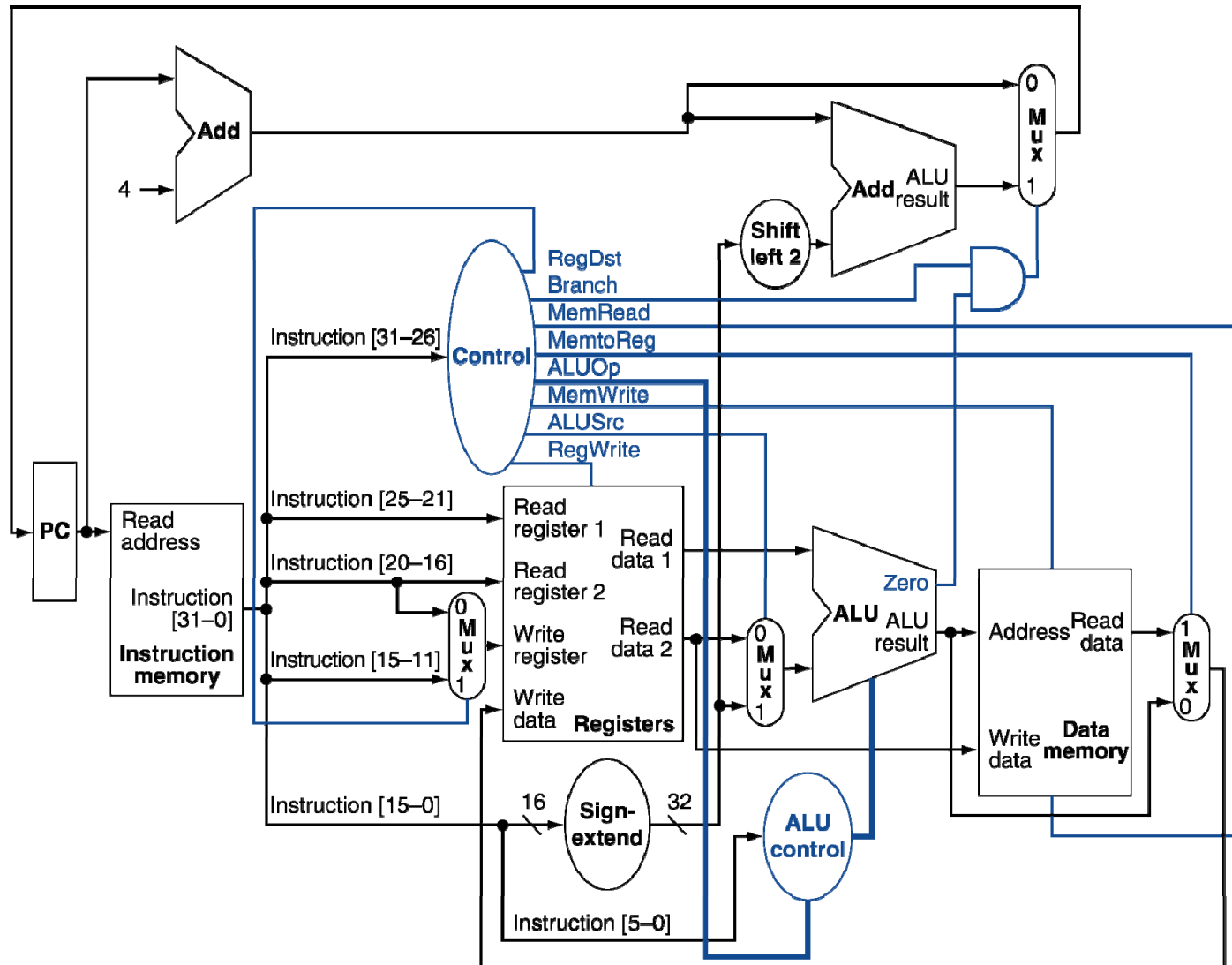
- $Y = F(A, B)$



A Recap: State Elements

- Registers
 - Data Memory
 - Instruction Memory
-
- Clocks are needed to decide when an element that contains state should be updated

Datapath With Control

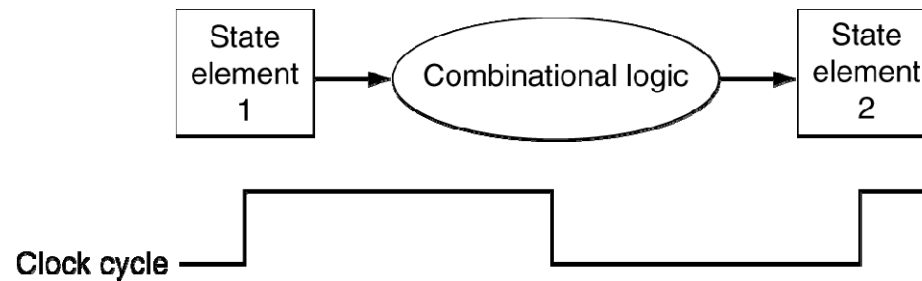


Clocking Methodology

- We study
 - Edge triggered methodology
 - Because it is simple
- Edge triggered methodology:
 - All state changes occur on a clock edge

Clocking Methodology

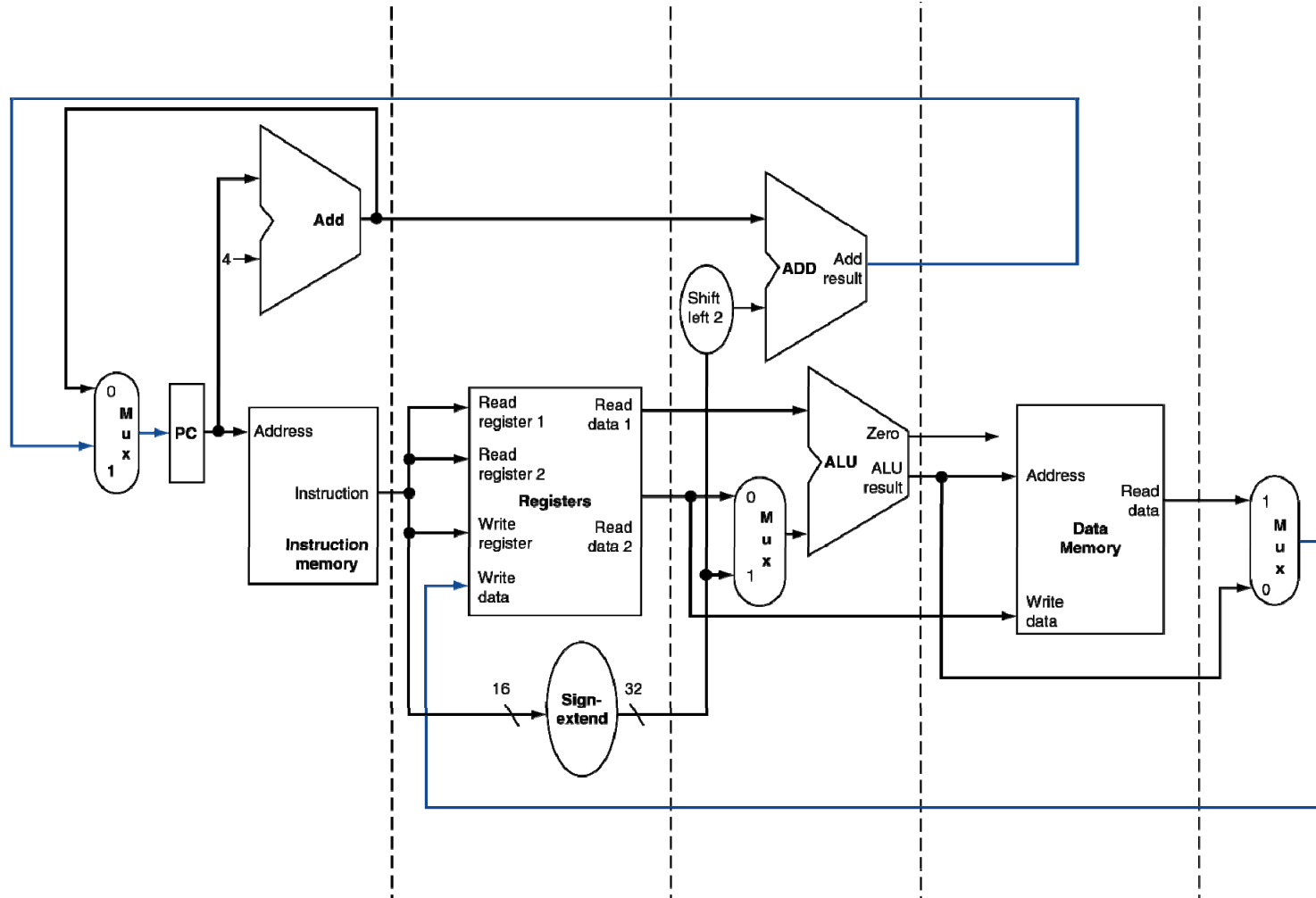
- Longest delay determines clock period



Single Cycle: Performance

- Assume time for stages is
 - 100ps for register read or write
 - 200ps for other stages
- What is the clock cycle time ?

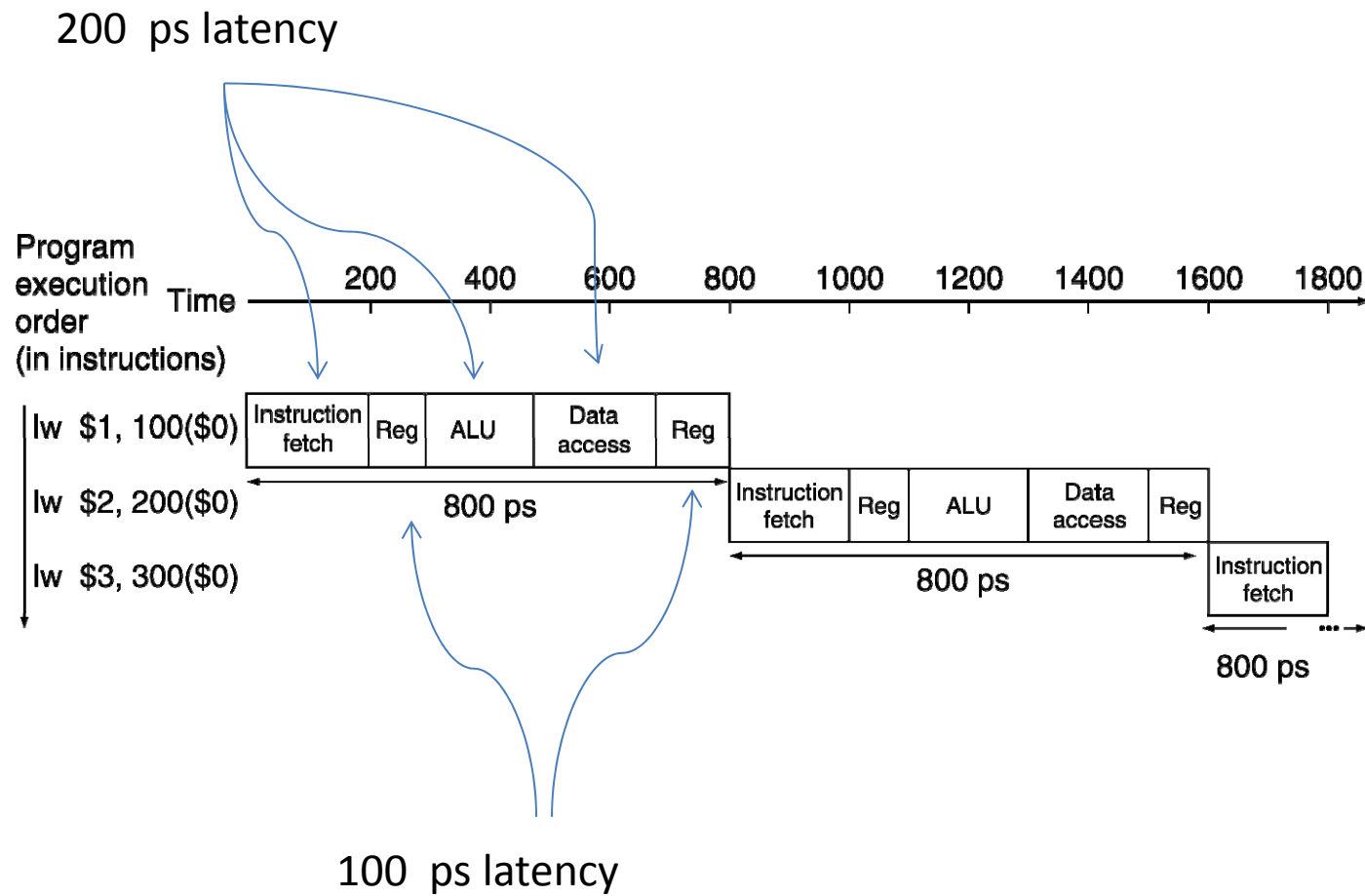
The 5 Stages



Single Cycle: Performance

- Assume time for stages is
 - 100ps for register read or write
 - 200ps for other stages

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps



Single Cycle: Performance

Performance Issues

- Longest delay determines clock period
 - Critical path: load instruction
 - Instruction memory → register file → ALU → data memory → register file

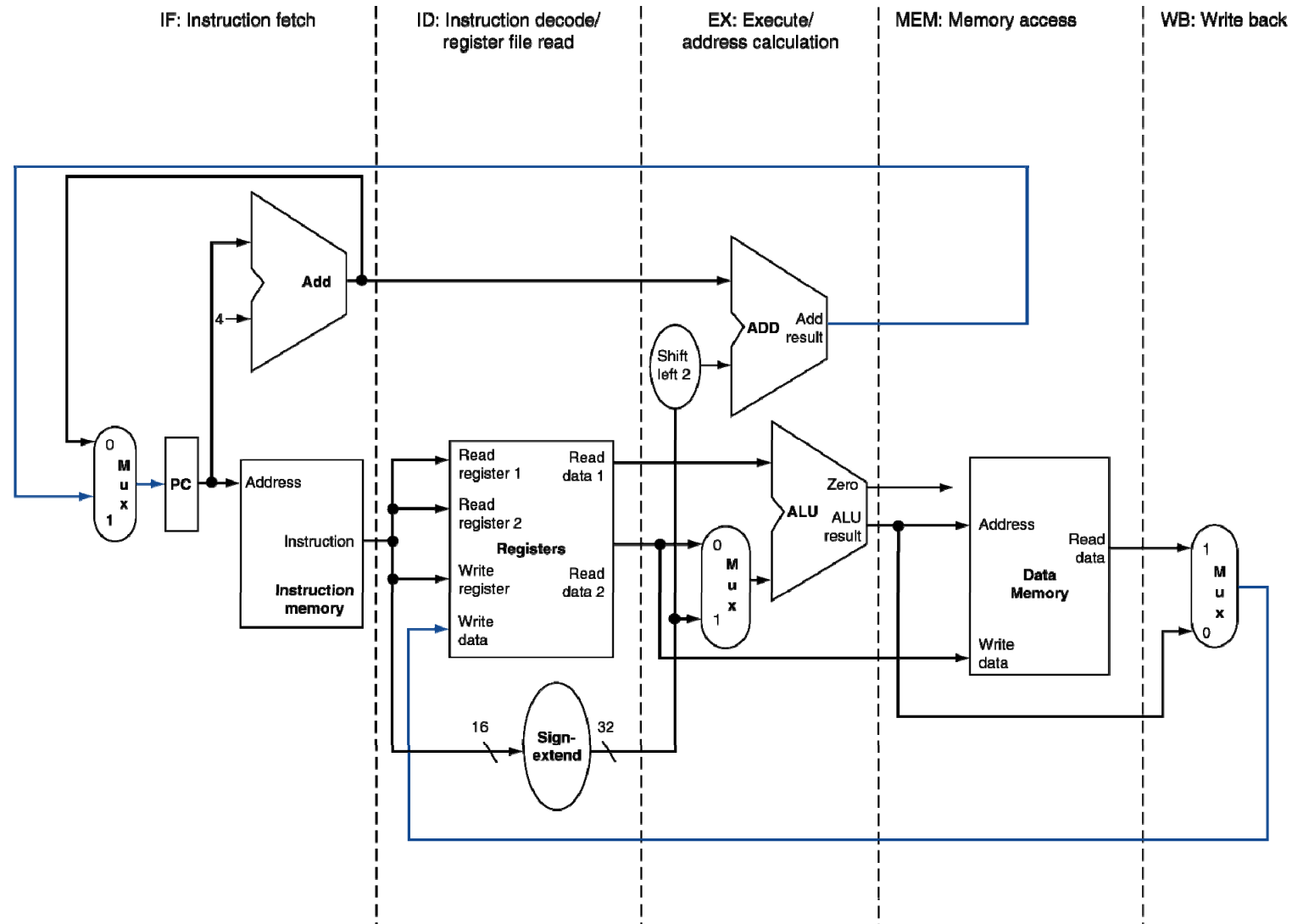
Performance Issues

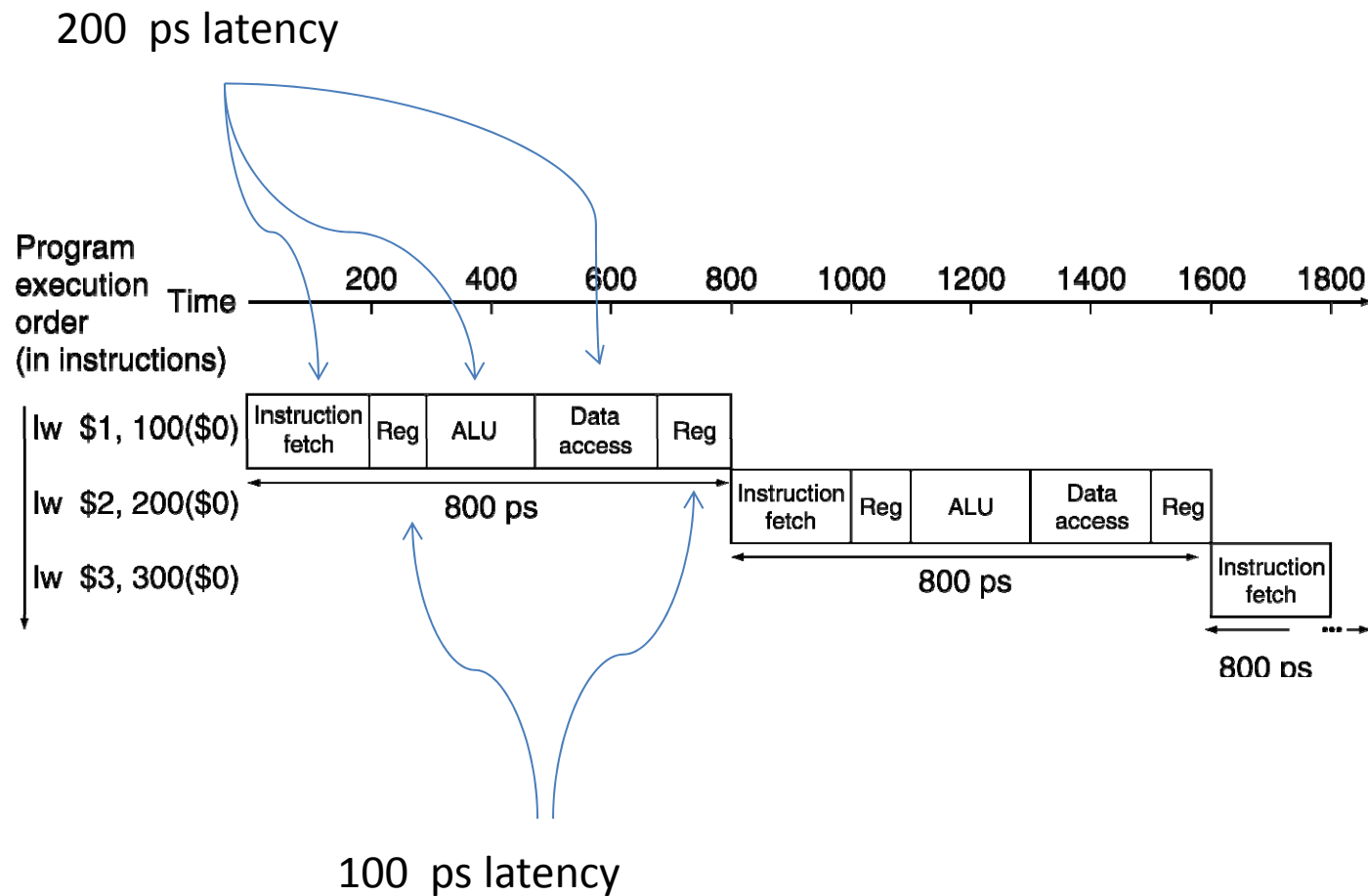
- Violates design principle
 - Making the common case fast
 - (read text in p.329-330)
- Improve performance by pipelining!

Recap: the stages of the datapath

- We can look at the datapath as five stages, one step per stage
 1. IF: Instruction fetch from memory
 2. ID: Instruction decode & register read
 3. EX: Execute operation or calculate address
 4. MEM: Access memory operand
 5. WB: Write result back to register

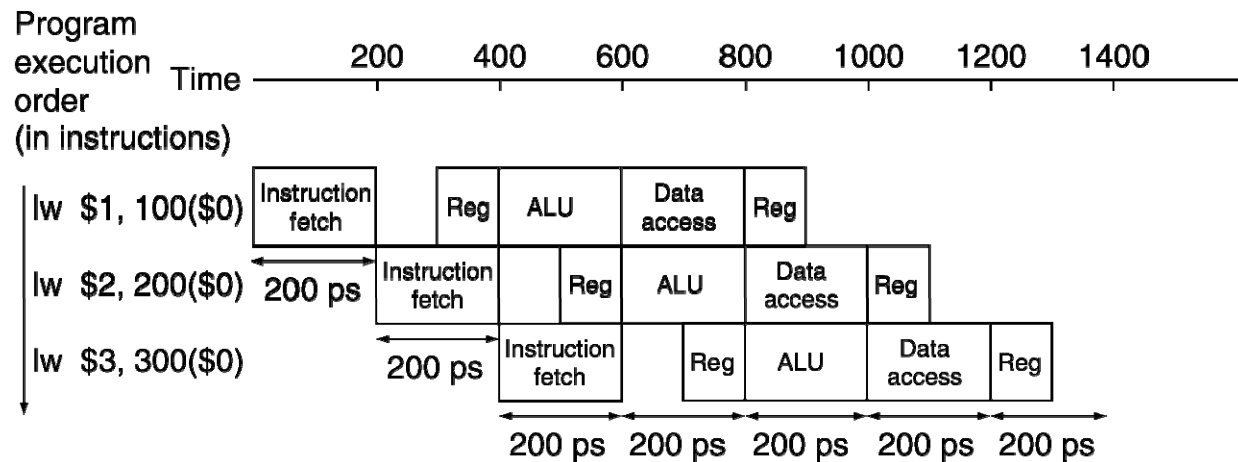
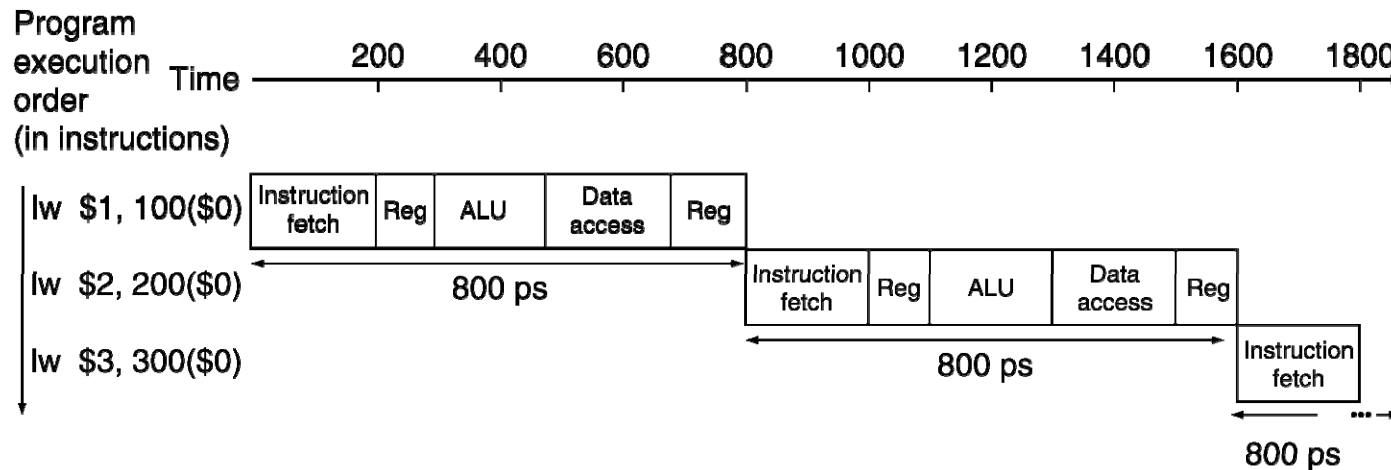
The 5 Stages





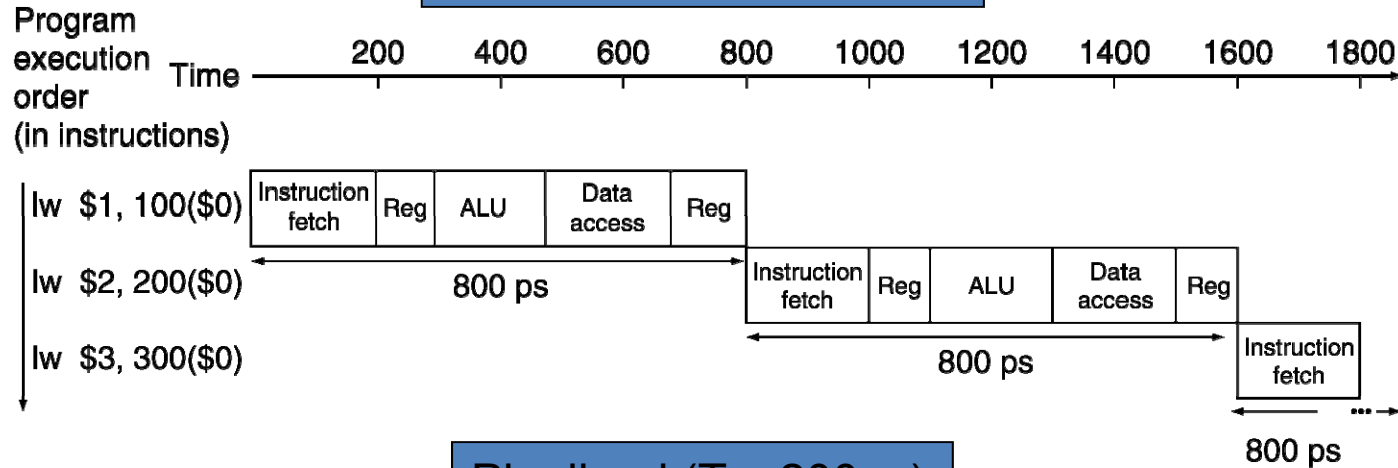
- Pipelining = Overlap the stages

Pipeline

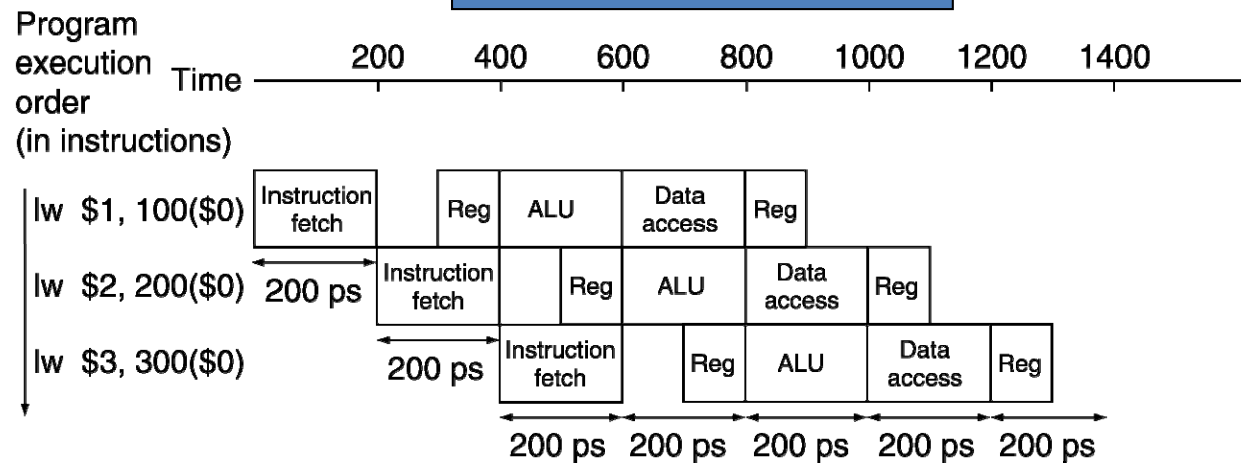


Clock Cycle

Single-cycle ($T_c = 800\text{ps}$)



Pipelined ($T_c = 200\text{ps}$)



- Even if some stages take only 100ps instead of 200ps, the pipelined execution clock cycle must have the worst case clock cycle time of 200ps

Speedup from Pipelining

- If all stages are balanced
 - i.e., all take the same time
 - $\text{Time between instructions}_{\text{pipelined}} = \frac{\text{Time between instructions}_{\text{nonpipelined}}}{\text{Number of stages}}$

Speedup from Pipelining

- If the stages are not balanced (not equal), speedup is less . Look at our example:
 - Time between instructions (non-pipelined) = 800ps
 - Number of stages = 5
 - Time between instructions (pipelined) = $800/5 = 160$ ps
 - But, what did we get ?
- Also read text in p.334

Speedup from Pipelining

- Recall from Chapter 1: Throughput versus latency
- Speedup in pipelining is due to increased throughput
 - Latency (time for each instruction) does not decrease

Hazards

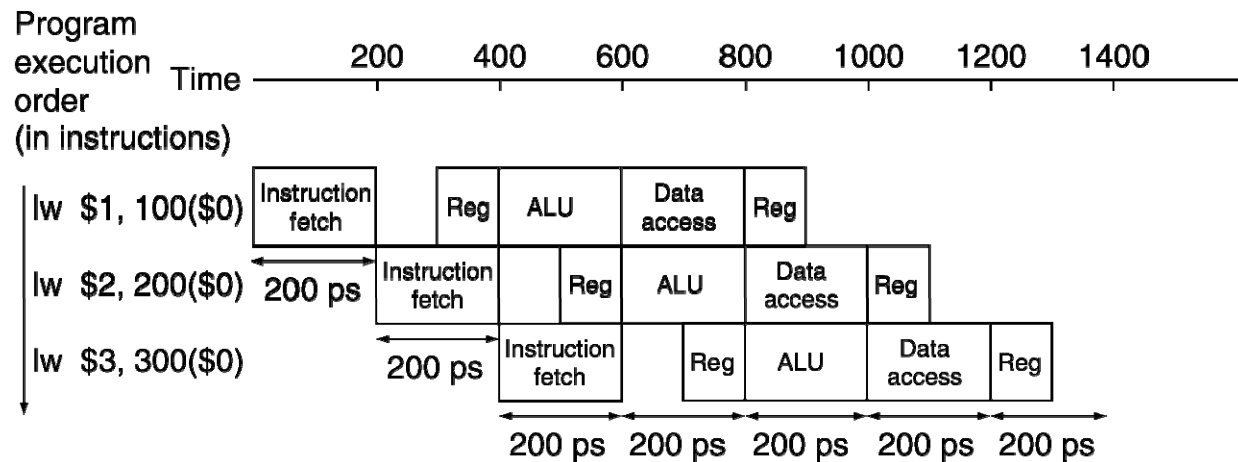
- Situations that prevent starting the next instruction in the next cycle are called hazards
- There are 3 kinds of hazards
 - Structure hazards
 - Data hazards
 - Control hazards

Structure Hazards

- When an instruction cannot execute in proper cycle due to a conflict for use of a resource

Example of structural hazard

- Imagine
 - a MIPS pipeline with a single data and instruction memory
 - a fourth additional instruction in the pipeline; when this instruction is trying to fetch from instruction memory, the first instruction is fetching of data memory – leading to a hazard
- Hence, pipelined datapaths require separate instruction/data memories
 - Or separate instruction/data caches

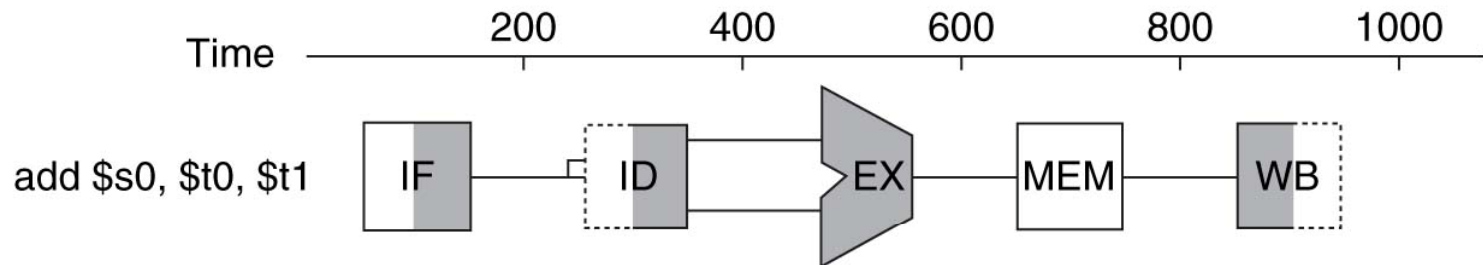


Data Hazards

- An instruction depends on completion of data access by a previous instruction

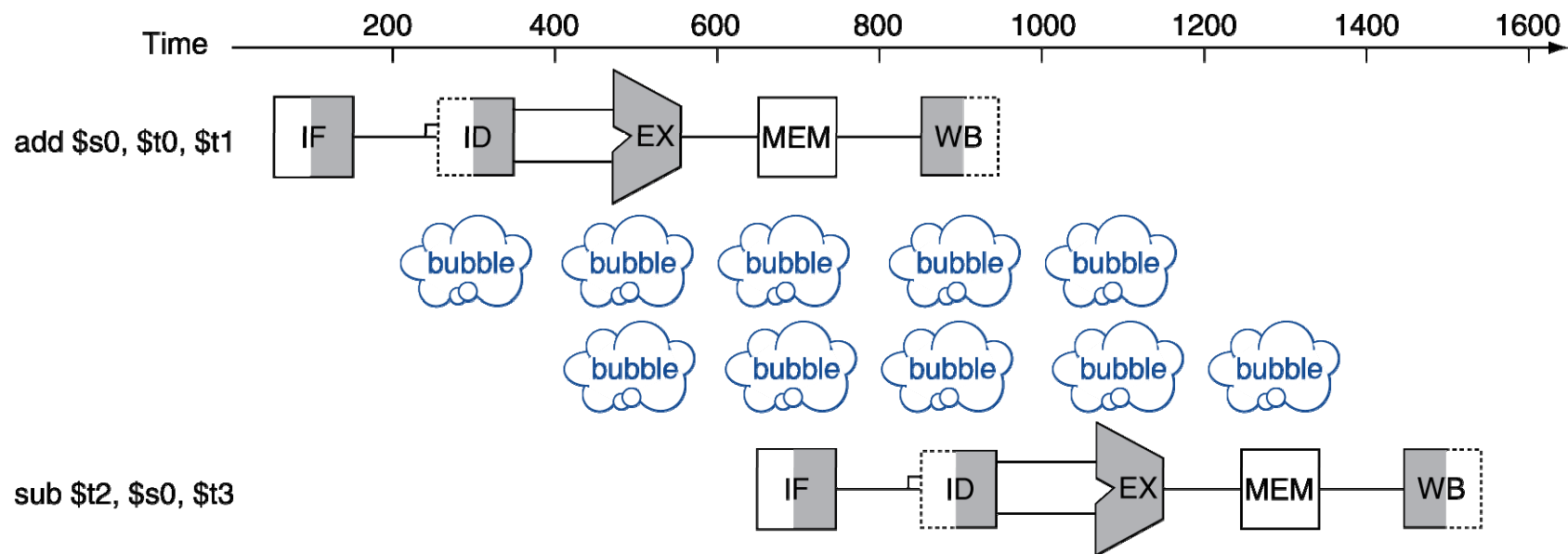
Data Hazards

- add **\$s0**, \$t0, \$t1
sub \$t2, **\$s0**, \$t3
- For the above code, when can we start the second instruction ?



Data Hazards

- add **\$s0**, \$t0, \$t1
sub \$t2, **\$s0**, \$t3



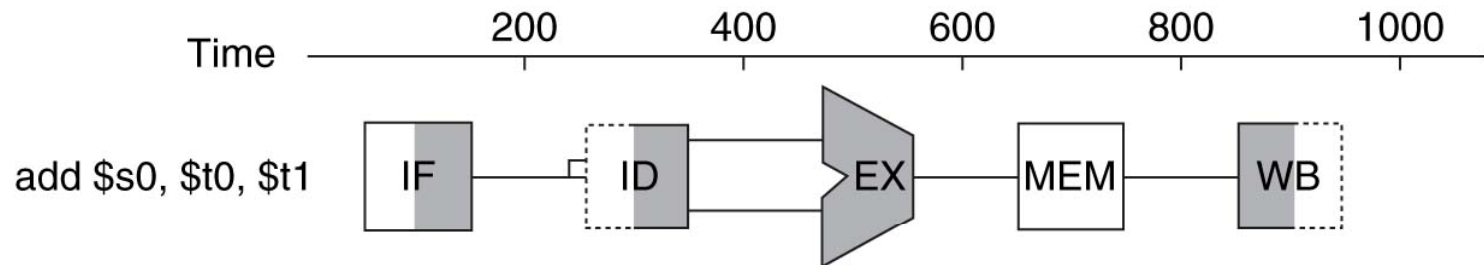
- Bubble or the pipeline stall is used to resolve a hazard
- BUT it leads to wastage of cycles = performance deterioration
- Solve the performance problem by 'forwarding' the required data

Forwarding (aka Bypassing)

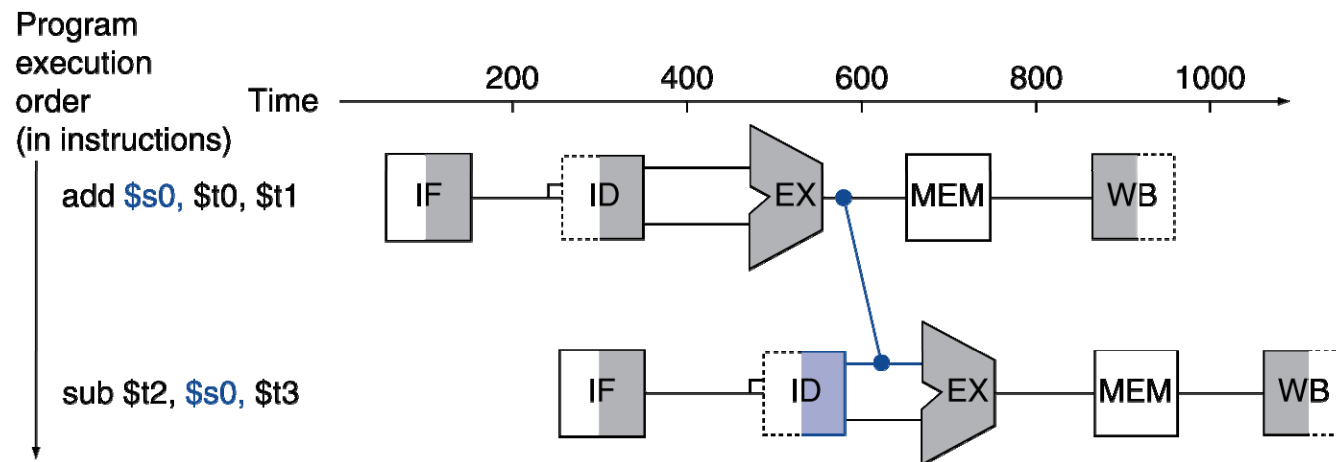
- Use result when it is computed : don't wait for it to be stored in a register
- Requires extra connections in the datapath

Forwarding (aka Bypassing)

- add **\$s0**, \$t0, \$t1
sub \$t2, **\$s0**, \$t3
- For the above code, draw the datapath with forwarding

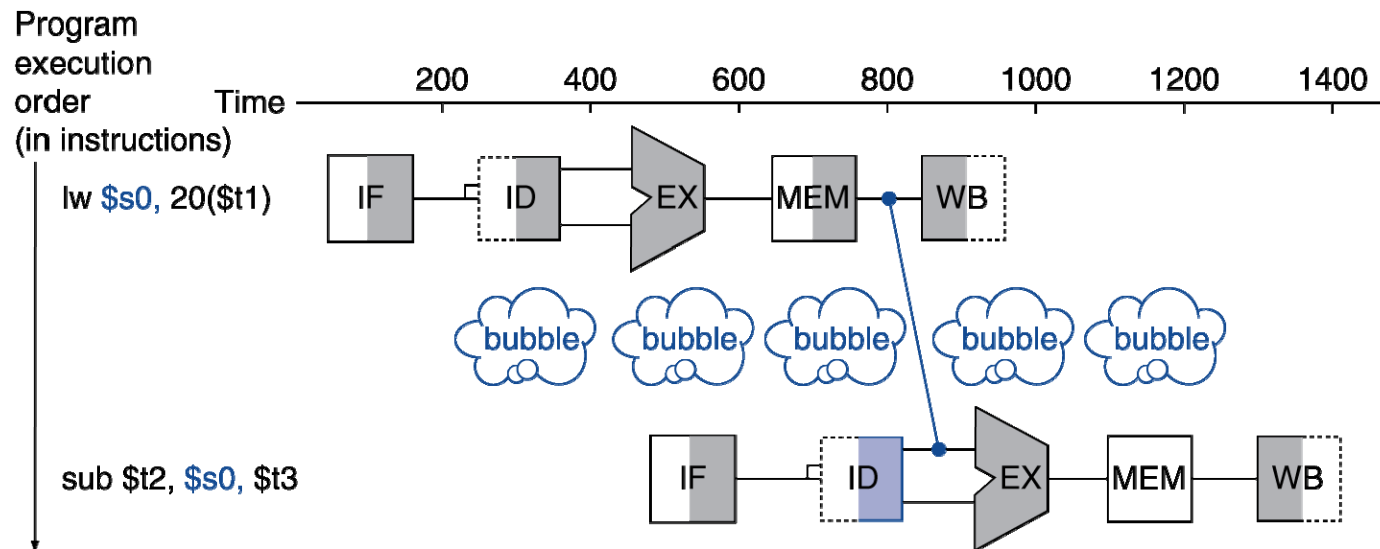


Forwarding (aka Bypassing)



Load-Use Data Hazard

- Can't always avoid stalls by forwarding
 - If value not computed when needed



Code Scheduling to Avoid Stalls

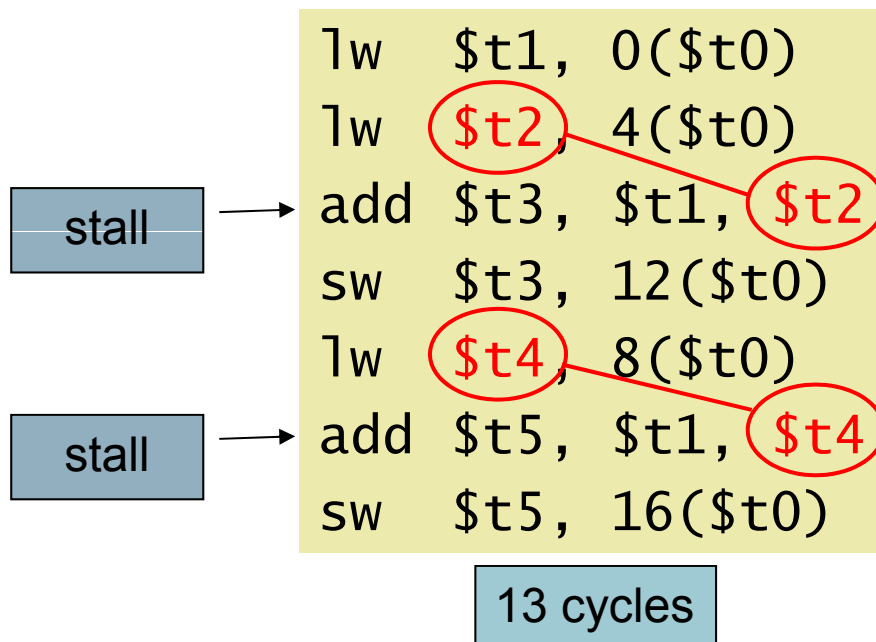
- Reorder code to avoid use of load result in the next instruction
- C code for $A = B + E; C = B + F;$

```
lw    $t1, 0($t0)
lw    $t2, 4($t0)
add   $t3, $t1, $t2
sw    $t3, 12($t0)
lw    $t4, 8($t0)
add   $t5, $t1, $t4
sw    $t5, 16($t0)
```

13 cycles

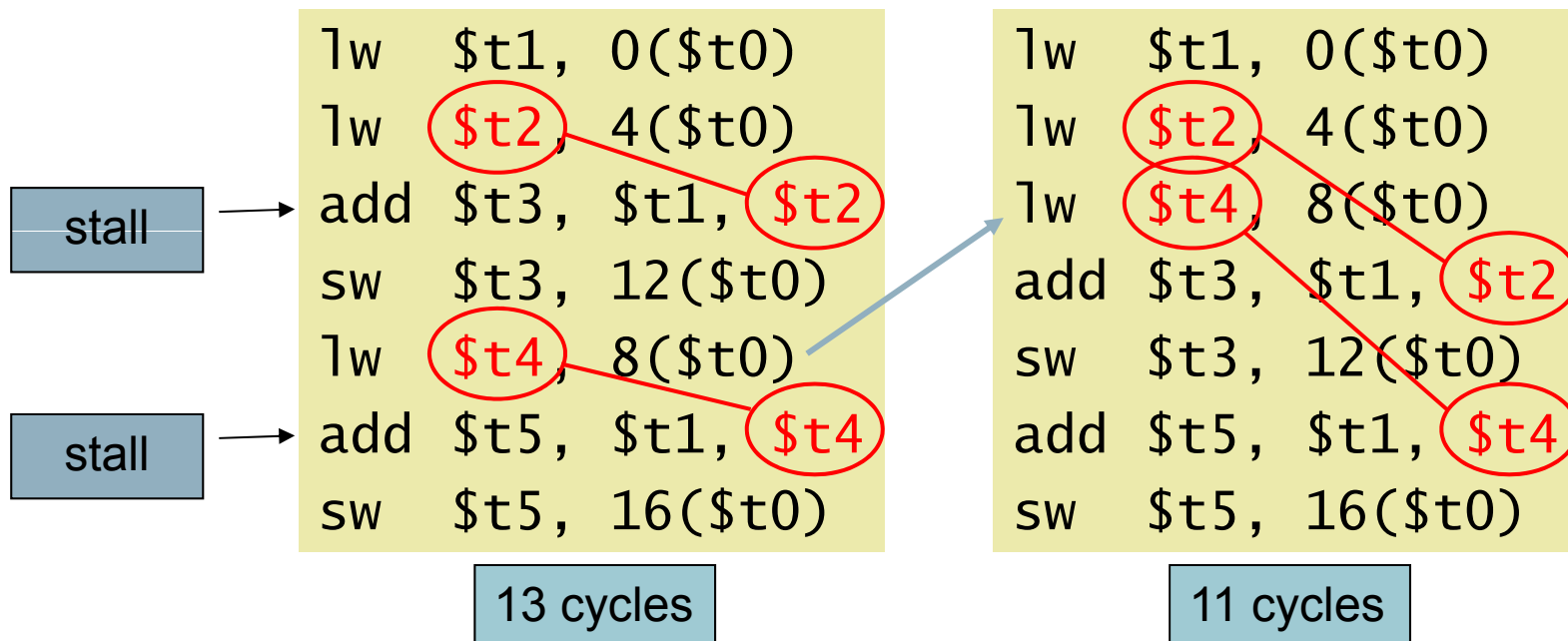
Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- C code for $A = B + E$; $C = B + F$;



Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- C code for $A = B + E$; $C = B + F$;



Control Hazards

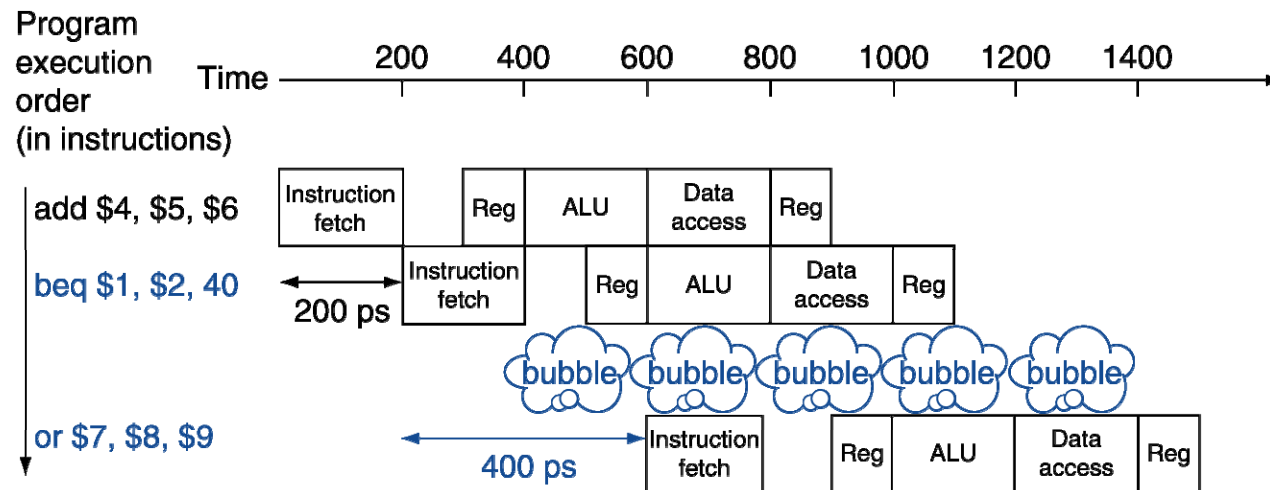
- When the proper instruction cannot execute in the proper pipeline clock cycle because the instruction that was fetched is not the one that is needed

Control Hazards

- Branch determines flow of control
 - Fetching next instruction depends on branch outcome
- Naïve/Simple Solution: Stall if we fetch a branch instruction
 - Drawback: Several cycles lost
 - Improve by adding hardware so that already in second stage we can know the address of the next instruction

Stall on Branch

- Wait until branch outcome determined before fetching next instruction
 - but with extra hardware to minimize the stalled cycles

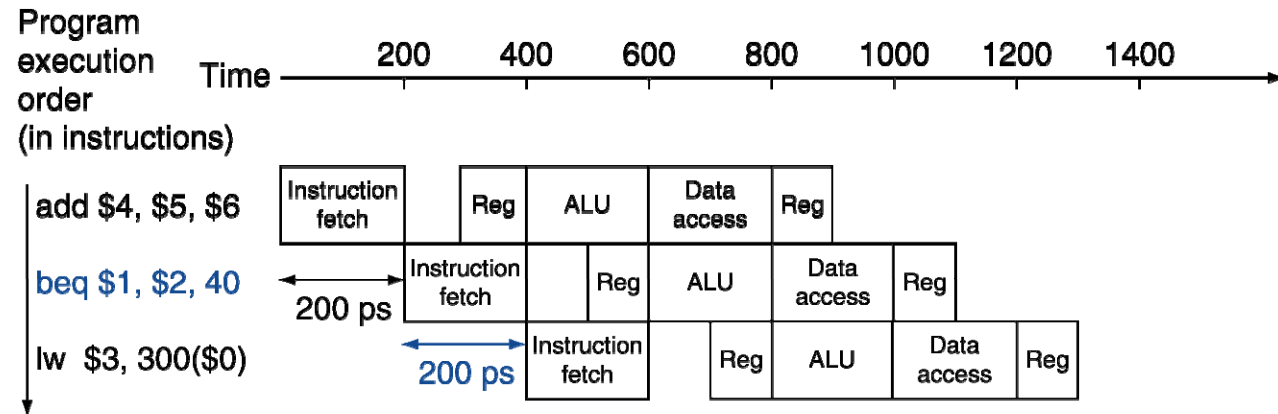


Branch Prediction

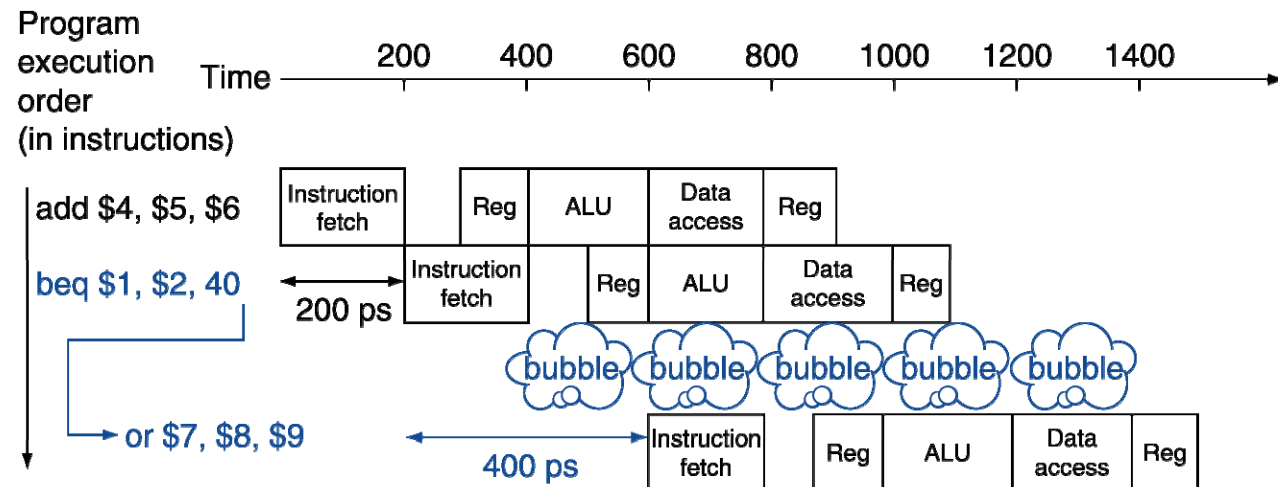
- Longer pipelines can't readily determine branch outcome early (in second stage)
 - Stall penalty becomes unacceptable
- Solution: predict outcome of branch
 - Only stall if prediction is wrong
 - We will not study the details

MIPS with Predict Not Taken

Prediction
correct



Prediction
incorrect



Recap: Hazards

- Situations that prevent starting the next instruction in the next cycle
- Structure hazards
 - A required resource is busy
- Data hazard
 - Need to wait for previous instruction to complete its data read/write
- Control hazard
 - Deciding on control action depends on previous instruction

Pipeline Summary

- Pipelining improves performance by increasing instruction throughput
 - Executes multiple instructions in parallel
 - Each instruction has the same latency
- Subject to hazards
 - Structure, data, control
- Instruction set design affects complexity of pipeline implementation (p.335)

Instruction-Level Parallelism (ILP)

- Pipelining: executing multiple instructions in parallel
- To increase ILP
 - Deeper pipeline
 - Less work per stage $\Rightarrow$ shorter clock cycle
 - Multiple issue
 - Replicate pipeline stages $\Rightarrow$ multiple pipelines
 - Start multiple instructions per clock cycle
 - $\text{CPI} < 1$, so use Instructions Per Cycle (IPC)
 - E.g., 4GHz 4-way multiple-issue
 - 16 BIPS, peak $\text{CPI} = 0.25$, peak $\text{IPC} = 4$
 - But dependencies reduce this in practice

Multiple Issue

- Static multiple issue
 - Compiler groups instructions to be issued together
 - Packages them into “issue slots”
 - Compiler detects and avoids hazards
- Dynamic multiple issue
 - CPU examines instruction stream and chooses instructions to issue each cycle
 - Compiler can help by reordering instructions
 - CPU resolves hazards using advanced techniques at runtime

Speculation

- “Guess” what to do with an instruction
 - Start operation as soon as possible
 - Check whether guess was right
 - If so, complete the operation
 - If not, roll-back and do the right thing
- Common to static and dynamic multiple issue
- Examples
 - Speculate on branch outcome
 - Roll back if path taken is different
 - Speculate on load
 - Roll back if location is updated

Compiler/Hardware Speculation

- Compiler can reorder instructions
 - Can include “fix-up” instructions to recover from incorrect guess
- Hardware can look ahead for instructions to execute
 - Buffer results until it determines they are actually needed
 - Flush buffers on incorrect speculation

Static Multiple Issue

- Compiler groups instructions into “issue packets”
 - Group of instructions that can be issued on a single cycle
 - Determined by pipeline resources required
- Think of an issue packet as a very long instruction
 - Specifies multiple concurrent operations
 - $\Rightarrow$ Very Long Instruction Word (VLIW)

Scheduling Static Multiple Issue

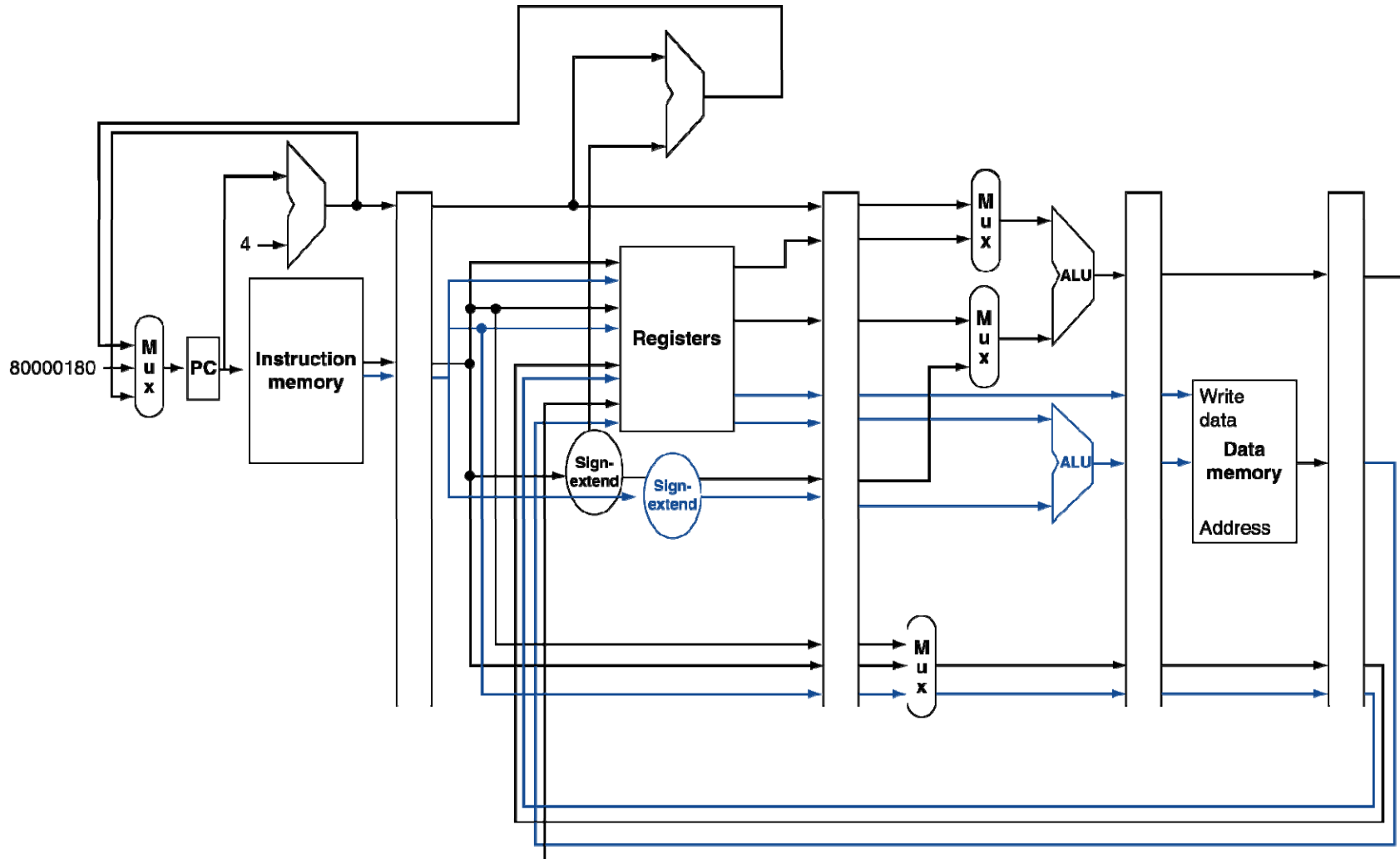
- Compiler must remove some/all hazards
 - Reorder instructions into issue packets
 - No dependencies with a packet
 - Possibly some dependencies between packets
 - Varies between ISAs; compiler must know!
 - Pad with nop if necessary

MIPS with Static Dual Issue

- Two-issue packets
 - One ALU/branch instruction
 - One load/store instruction
 - 64-bit aligned
 - ALU/branch, then load/store
 - Pad an unused instruction with nop

Address	Instruction type	Pipeline Stages						
		IF	ID	EX	MEM	WB		
n	ALU/branch	IF	ID	EX	MEM	WB		
n + 4	Load/store	IF	ID	EX	MEM	WB		
n + 8	ALU/branch		IF	ID	EX	MEM	WB	
n + 12	Load/store		IF	ID	EX	MEM	WB	
n + 16	ALU/branch			IF	ID	EX	MEM	WB
n + 20	Load/store			IF	ID	EX	MEM	WB

MIPS with Static Dual Issue



Hazards in the Dual-Issue MIPS

- More instructions executing in parallel
- Data hazard
 - Forwarding avoided stalls with single-issue
 - Now can't use ALU result in load/store in same packet
 - add `$t0`, `$s0`, `$s1`
load `$s2`, 0(`$t0`)
 - Split into two packets, effectively a stall
- Load-use hazard
 - Still one cycle use latency, but now two instructions
- More aggressive scheduling required

Dynamic Multiple Issue

- “Superscalar” processors
- CPU decides whether to issue 0, 1, 2, ... each cycle
 - Avoiding structural and data hazards
- Avoids the need for compiler scheduling
 - Though it may still help
 - Code semantics ensured by the CPU

Dynamic Pipeline Scheduling

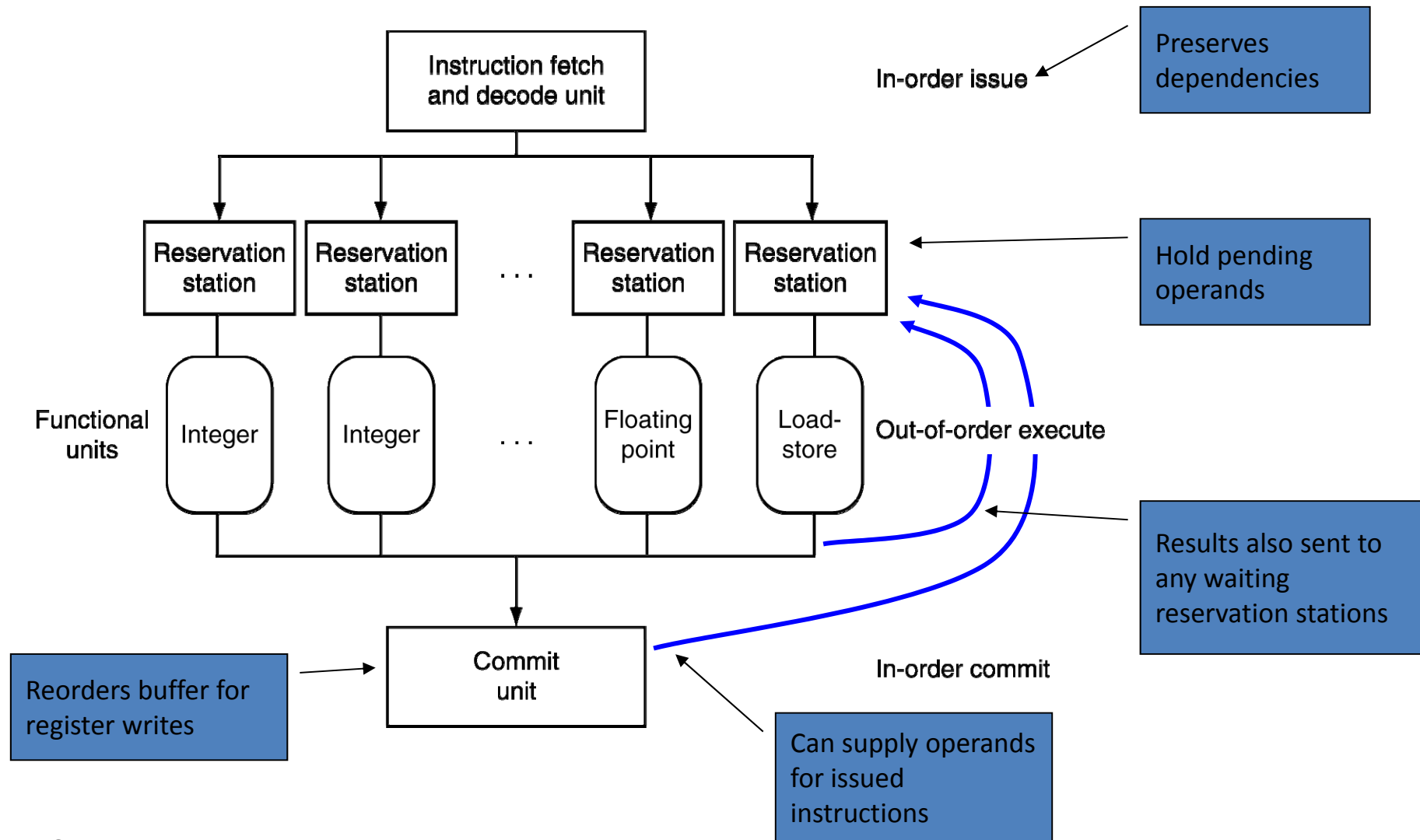
- Allow the CPU to execute instructions out of order to avoid stalls
 - But commit result to registers in order

- Example

```
lw      $t0, 20($s2)
addu    $t1, $t0, $t2
sub      $s4, $s4, $t3
slti    $t5, $s4, 20
```

- Can start sub while addu is waiting for lw

Dynamically Scheduled CPU



Why Do Dynamic Scheduling?

- Why not just let the compiler schedule code?
- Not all stalls are predicable
 - e.g., cache misses
- Can't always schedule around branches
 - Branch outcome is dynamically determined
- Different implementations of an ISA have different latencies and hazards

Does Multiple Issue Work?

- Yes, but not as much as we'd like
- Programs have real dependencies that limit ILP
- Some dependencies are hard to eliminate
- Some parallelism is hard to expose
 - Limited window size during instruction issue
- Memory delays and limited bandwidth
 - Hard to keep pipelines full
- Speculation can help if done well